



Quantifying AWS S3 I/O Performance Boundaries Using the Roofline Model

Meng Tang*
Illinois Institute of Technology
Chicago, USA
mtang11@hawk.illinoistech.edu

Zhaobin Zhu*
Johannes Gutenberg University Mainz
Mainz, Germany
zhu@uni-mainz.de

Luanzheng Guo
Pacific Northwest National
Laboratory (PNNL)
Livermore, USA
lenny.guo@pnnl.gov

James G. Bandy
Pacific Northwest National
Laboratory (PNNL)
Richmond, USA
james.bandy@pnnl.gov

Tim Carlson
Pacific Northwest National
Laboratory (PNNL)
Mainz, USA
timothy.carlson@pnnl.gov

Sarah Neuwirth
Johannes Gutenberg University Mainz
Mainz, Germany
Julich Supercomputing Centre
Mainz, Germany
neuwrith@uni-mainz.de

Anthony Kougkas
Illinois Institute of Technology
Chicago, USA
akougkas@illinoistech.edu

Xian-He Sun
Illinois Institute of Technology
Chicago, USA
sun@illinoistech.edu

Nathan R. Tallent
Pacific Northwest National
Laboratory (PNNL)
Richland, USA
tallent@pnnl.gov

Abstract

High-performance computing facilities increasingly form hybrid environments that integrate cloud services. To avoid cumbersome network transfers when sharing data, a new class of storage gateways map a subset of facility storage to a cloud counterpart and automatically manage data mirroring. However, the performance characteristics of accessing AWS's S3 from HPC systems using different methods and patterns remains poorly understood. This paper presents a roofline-based analysis of three S3 integration approaches: NFS-mounted AWS Storage Gateway, data migration through Storage Gateway, and direct S3 API transfers. We extend I/O roofline modeling to characterize operational intensity and bandwidth ceilings across varying data sizes and access patterns. Our experimental evaluation demonstrates significant performance differences between access methods, with POSIX I/O on NFS Storage Gateway achieving up to 6.4× higher bandwidth than other approaches for large transfers. The roofline analysis reveals distinct characteristics for each method, enabling informed selection of S3 integration strategies.

CCS Concepts

• **Information systems** → **Information storage technologies**;
• **Networks** → *Network performance evaluation*; **Network protocols**; **Cloud computing**; • **Software and its engineering** →

* Authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SC Workshops '25, St Louis, MO, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1871-7/25/11
<https://doi.org/10.1145/3731599.3767513>

Input / output; • General and reference → Cross-computing tools and techniques.

Keywords

AWS S3, distributed workflows, performance analysis, storage bottlenecks, I/O monitoring, I/O roofline model

ACM Reference Format:

Meng Tang, Zhaobin Zhu, Luanzheng Guo, James G. Bandy, Tim Carlson, Sarah Neuwirth, Anthony Kougkas, Xian-He Sun, and Nathan R. Tallent. 2025. Quantifying AWS S3 I/O Performance Boundaries Using the Roofline Model. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3731599.3767513>

1 Introduction

High Performance Computing (HPC) systems are increasingly adopting hybrid cloud services to leverage virtually unlimited storage capacity and computational elasticity [8, 15, 16]. This convergence of traditional HPC infrastructure with cloud services has introduced new I/O paradigms, particularly in data-intensive scientific workloads that require access to remote cloud storage systems such as Amazon Web Services (AWS) Simple Storage Service (S3) [18]. However, the performance characteristics of accessing cloud storage from HPC environments remain poorly understood, creating challenges for I/O optimization and resource allocation decisions.

The performance of cloud storage access from HPC systems is influenced by multiple factors including transfer protocols, data access patterns, parallelism levels, and network topology [17]. AWS provides multiple pathways for accessing S3 storage—including NFS-compatible interfaces through Storage Gateway and direct API-based transfers—yet the relative performance implications of

these approaches under different workload conditions remain unclear [3]. AWS Storage Gateway enables traditional HPC applications to access cloud storage without modification through familiar POSIX interfaces, but the performance trade-offs compared to direct API-based transfers are poorly understood in HPC environments.

Existing work has evaluated AWS S3 as a storage backend for HPC applications [1, 9]; however, these studies primarily focused on techniques that convert typical HPC POSIX operations to S3 operations without providing comprehensive performance comparisons across different S3 access methodologies. The lack of systematic evaluation comparing traditional HPC I/O patterns through AWS Storage Gateway versus direct S3 API usage makes it difficult to understand performance trade-offs and identify optimal strategies for different use cases. This gap in comparative analysis hinders informed decision-making about cloud storage integration in data-intensive scientific computing environments.

To address this gap, we present the first I/O roofline analysis of AWS S3 performance from HPC systems. Building on the established roofline model for understanding performance capabilities and limitations of parallel systems [19], the I/O roofline framework provides a visual methodology for storage system characterization that does not require deep knowledge of the underlying I/O stack architecture [20]. This approach enables systematic performance comparison and bottleneck identification, providing actionable insights for cloud storage integration in data-intensive scientific computing environments.

The main contributions of this work are:

- We extend I/O roofline modeling for characterizing AWS S3 performance from HPC systems across three access methods: NFS-mounted Storage Gateway, data migration through Storage Gateway, and direct S3 API transfers.
- We systematically compare S3 access strategies, analyzing parallel POSIX I/O, data movement efficiency, and direct API performance across varying data sizes and parallelism levels.
- We identify performance bottlenecks and optimization opportunities for each method, providing quantitative guidance for S3 integration with performance differences up to 6.4x between approaches.

2 Background and Related work

2.1 Cloud Storage in HPC Environments

The increasing adoption of hybrid cloud-HPC architectures has driven significant interest in integrating cloud storage services with traditional high-performance computing environments [12, 14]. This integration presents fundamental challenges including variable network latency, bandwidth limitations, and consistency models that may not align with HPC performance expectations [11, 13]. As cloud platforms have evolved to support HPC workloads through specialized instances and high-speed networking, understanding different storage access methodologies under realistic HPC conditions has become critical. AWS S3 provides scalable object storage through REST APIs, while AWS Storage Gateway offers hybrid cloud storage services connecting on-premises environments to AWS cloud storage via standard protocols such as NFS [3]. Cloud storage performance is influenced by virtualization overhead, network protocol efficiency, and I/O parallelism.

Recent evaluations have highlighted these challenges, with [10] demonstrating that current S3 systems fall short of HPC requirements, particularly in scalability and latency when tested with HPC benchmarks such as IO500 and MD-Workbench. Specialized optimizations such as S3Embedded have been developed to bridge this performance gap [10].

2.2 Roofline Model

The Roofline model [19] is a widely used approach for node-level performance analysis based on bound and bottleneck analysis. It visualizes performance limits and potential bottlenecks in parallel computing systems by presenting operational intensity (FLOPS/byte) in relation to achievable performance (FLOPS). The x-axis represents operational intensity, the ratio of floating-point operations to memory bytes accessed, while the y-axis shows maximum performance.

Equation 1 defines the *maximum attainable performance (AP)*, which depends on hardware constraints such as memory bandwidth and compute power:

$$AP \text{ (GFlops/s)} = \min(\text{Peak Floating Point Performance, Peak Memory Bandwidth} \times \text{OI}) \quad (1)$$

A key metric is *operational intensity (OI)* (Equation 2), which balances computation and memory access. Higher intensity corresponds to fewer memory transfers per operation, while lower intensity indicates more memory-bound workloads.

$$\text{Operational intensity} = \frac{\text{Floating point operations}}{\text{Memory bytes transferred}} \quad (2)$$

An algorithm is *compute-bound* if performance hits the flat Roofline or *memory-bound* if limited by the sloped Roofline. The ridge point (RP), where these lines intersect, indicates the optimal operational intensity for a given system.

Overall, the Roofline model provides an intuitive way to analyze application performance, helping identify bottlenecks and optimize system efficiency. Regarding its simplicity and ease of use, many works have extended or modified it to cover more aspects like I/O, communication, or even energy.

2.2.1 Workflow Roofline Model. [7] extended the Roofline Model to analyze full end-to-end scientific workflows. It maps each step's performance and arithmetic intensity against system-level bandwidth limits, demonstrating how different workflows hit different bottlenecks, e.g., I/O, node bandwidth, or data movement, thus helping identify optimization opportunities. However, this approach remains focused on computational workflows and does not address storage system characterization.

2.2.2 Communication and Network-Aware Roofline Models. [4] propose an extension to the Roofline model that incorporates communication costs for distributed-memory HPC systems. Their communication aware Roofline model introduces communication arithmetic intensity and evaluates attainable performance while accounting for communication bandwidth constraints. Peak communication performance is measured using the *Ping-Pong* micro-benchmark. This extended model introduces an additional dimension for communication, enabling both 2D and 3D visualizations. Similarly, the *Ridgeline*

model [5] offers a simplified planar representation of Roofline diagrams when analyzing multiple system resources. However, both approaches focus primarily on computation, memory, and network communication, and do not address storage I/O performance or the complexities of cloud storage access patterns that are increasingly important in hybrid HPC environments.

2.2.3 I/O Roofline Model. [20, 21] present an empirical I/O Roofline model for evaluating and comparing I/O performance in HPC systems. It extends the traditional Roofline model to include I/O metrics like bandwidth and IOPS, and introduces a scoring method to quantify how close applications or systems come to their peak I/O performance. The model is tool-independent, visual, and tested on real systems using benchmarks and profiling tools like IOR and Darshan. [6] introduces a Roofline Model of Energy, which extends the traditional Roofline approach by using energy efficiency (FLOPs per joule) as the performance metric instead of execution speed. This enables comparing applications and architectures based on of energy use, not just throughput. However, existing I/O Roofline implementations focus exclusively on traditional parallel file systems using POSIX and MPI-IO interfaces, lacking the capability to characterize heterogeneous cloud storage access methods such as NFS gateways, data migration services, and direct API calls that are essential for hybrid cloud-HPC environments.

3 Methodology

To enable informed decision-making about AWS S3 integration in HPC environments, we conduct a targeted roofline-based performance comparison of three distinct S3 access methodologies: NFS-mounted Storage Gateway, data migration through Storage Gateway, and direct API-based transfers.

3.1 AWS S3 Usage

Method 1: AWS Storage Gateway NFS Mount for Parallel I/O. This method evaluates the performance of parallel, file-per-task I/O operations using a standard HPC benchmark. We deploy AWS Storage Gateway in File Gateway mode to provide NFS-compatible access to S3 storage from HPC compute nodes. The **Storage Gateway** at PNNL is configured with 300 GB of local cache storage to optimize frequently accessed data and reduce latency for repeated operations. The gateway is mounted on HPC compute nodes using the NFSv3 protocol across all allocated nodes through automated SSH-based deployment scripts; system specifications are shown in Table 2. Our implementation varies parallelism levels and tests multiple I/O transfer sizes (size of a single I/O operation) ranging from 4KB to 100MB with fixed block sizes (total I/O size per task) of 100MB to 1000MB per task (Table 1). The IOR benchmark is executed using POSIX API calls, cache clearing between runs, and data verification, while Darshan I/O tracing captures detailed access patterns and timing information. Three trials are conducted for each configuration to ensure statistical reliability.

Method 2: Data Migration via Storage Gateway. This benchmark evaluates bulk data transfer performance between the HPC cluster's BeeGFS storage and S3 via the NFS-mounted Storage Gateway. We generate test files equal to the number of parallel tasks, then perform bidirectional transfers using standard cp commands

distributed across compute nodes via SSH-based round-robin scheduling. Upload operations copy files from BeeGFS to the Storage Gateway path, while download operations retrieve data from the gateway back to BeeGFS. Each transfer is executed in parallel across multiple nodes with timing measurements, bandwidth calculations, and file integrity verification.

Method 3: Direct S3 API Implementation. We develop a custom C++ benchmark using AWS SDK for C++ with MPI parallelization to evaluate native S3 access performance [2]. The benchmark implements upload and download operations using configurable chunk sizes (1MB to 100MB), automatically selecting single-part uploads for files <5MB and multipart uploads for larger transfers. The multipart process follows three phases: (1) initiate upload to obtain an upload ID, (2) upload individual parts while tracking ETags, and (3) complete upload by combining all parts. Each process uses unique test files and distinct S3 object keys to avoid contention.

For both Methods 1 and 2, we account for the asynchronous nature of Storage Gateway's S3 synchronization by monitoring when data actually appears in the S3 bucket using AWS API calls. The time difference between local write completion and S3 object availability represents the gateway's upload latency, which we use to adjust the measured bandwidth calculations to reflect true end-to-end data transfer performance to cloud storage rather than just local cache performance.

3.2 Experimental Environment

Our experiments are conducted on a PNNL HPC cluster whose hardware specifications are detailed in Table 2. The cluster provides a theoretical maximum network bandwidth of approximately 1250 MB/s and connects to the AWS Storage Gateway through NFS mounts. All results are averaged over 3 runs.

Table 1: Experimental Configuration Parameters

Parameter	Method 1 & 2	Method 3
Nodes	1, 2, 4, 8, 10, 16	1, 2, 4, 8, 10
Tasks/Node	1, 2, 4, 8, 16, 32	1, 2, 4, 8, 16, 32
File Sizes	4K, 10M, 100M, 1000M	1M, 50M, 100M
Transfer Sizes	4K, 8K, 1M, 4M, 100M	AWS multipart (≥ 5 MB)

Table 2: PNNL HPC Cluster Hardware Specifications

Component	Specification
CPU	2× AMD EPYC 7502 @ 2.5 GHz (64 cores total)
Memory	264 GB DDR4 (4 GB per core)
Network	10 Gigabit Ethernet (MTU 1500)
Shared Storage	BeeGFS parallel file system

3.3 Roofline Model for S3 I/O Access

Typically for a Roofline model, the x-axis represents operational intensity (operations per unit of data), while the y-axis shows performance measured in operations per second. Since access to Amazon S3 can occur in various ways, i.e., not just through standard POSIX read and write operations, the existing I/O roofline model cannot be directly applied for our analysis. Instead of treating classical I/O operations (IOPs) as "work," we propose a more abstract definition of work, i.e., every I/O action (e.g., read, write, put, get, copy, etc.) is counted as an operation, regardless of its interface or implementation details. Accordingly, the maximum attainable performance

in term of I/O operations per second (**IOPS**) can be defined using Equation 3:

$$\text{Attainable Performance (IOPS)} = \frac{\text{Peak IOPS}}{\text{Peak I/O Bandwidth} \times \text{IOI}} \quad (3)$$

Since every I/O operation has an associated transfer size (e.g., the count parameter for POSIX read/write, the file size in a copy operation, or the chunk size during an upload) the attainable performance can be calculated based on the bandwidth and transfer size using Equation 4. For example, if a system can move 100 MB/s and each operation handles 1 MB, then it can perform 100 operations per second.

$$\text{IOPS} = \frac{\text{Bandwidth}}{\text{Transfer Size}} \quad (4)$$

For I/O operational intensity (**IOI**), which describes how many operations are performed per byte of data moved, we relate the I/O operations to the number of bytes transferred. Therefore, this can be defined using the standard approach shown in Equation 5, or alternatively derived as shown in Equation 6.

$$\text{IOI} = \frac{\text{IOPS}}{\text{Bytes transferred}} \quad (5)$$

$$\text{IOI} = \frac{\text{IOPS}}{\text{I/O Bandwidth}} \quad (6)$$

Based on the definition, smaller transfer sizes per I/O operation lead to higher I/O operational intensity, since more operations are needed to move a given amount of data. Conversely, larger transfers per operation result in lower I/O operational intensity. In general, we define work simply as an abstract (I/O) operation, regardless of the interface, access method, or data volume involved. This approach lets us consistently compare the performance of different I/O access methods across all scales, i.e., from individual node to full system.

4 Evaluation

Our evaluation compares and quantifies the performance characteristics and bottleneck behavior of three S3 access methodologies. We first present raw performance characteristics across varying experimental parameters to establish baseline behavior and identify optimal throughput configurations for I/O roofline analysis. We then apply roofline analysis to interpret results and identify operational intensity regimes where each approach achieves optimal performance.

4.1 Performance Scaling Analysis

We conduct comprehensive performance evaluations across multiple scaling dimensions for AWS Storage Gateway (Methods 1 and 2) and direct S3 API access (Method 3).

Figure 1a shows how increasing task-level parallelism from 1 to 32 tasks per node on a fixed 10-node configuration with 100MB files affects *Method 1*. We observe that 320 parallel tasks across 10 nodes achieve approximately 1120 MiB/sec for write operations and 1105 MiB/sec for read operations. As we can see, the performance starts reaching near-optimal bandwidth starting with a total of 80 tasks on 10 nodes.

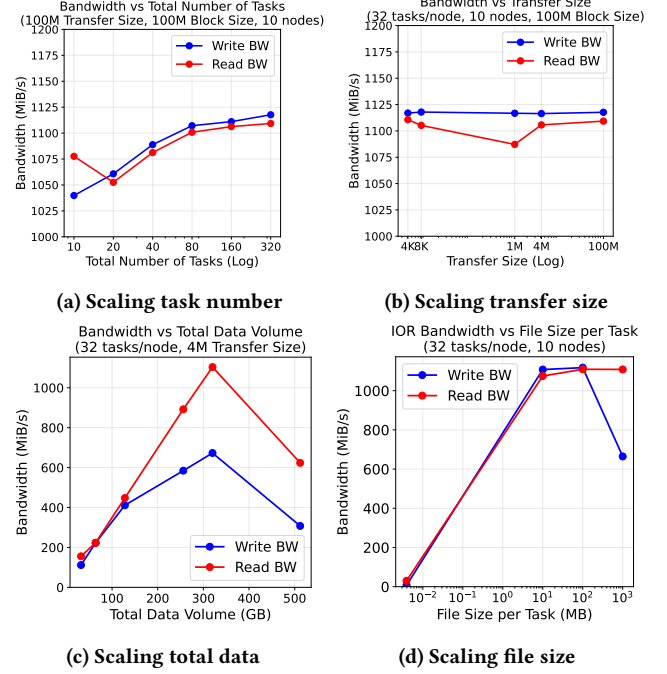


Figure 1: Performance analysis of POSIX I/O (Method 1) on AWS Gateway

The Figure 1b experiment examines the impact of I/O transfer granularity on Storage Gateway performance by varying transfer sizes from 4KB to 100MB while keeping all other parameters constant (10 nodes, 32 tasks, 100MB files). When maintaining constant total I/O size, we observe that transfer size does not significantly impact write performance, but read operations achieve optimal performance with 4KB and 100MB transfer sizes. The 1MB transfer size shows slightly worse read performance, though this represents only 5.85% variation, which is a reasonable variation range.

Figure 1c examines how performance scales with increasing total data volume by varying the number of nodes from 1 to 16 while maintaining a constant per-task workload (1000MB files, 32 tasks per node). For IOR operations, we use a fixed 4MB transfer size, as previous analysis (Figure 1b) demonstrates minimal performance variation across different transfer granularities. We observe that performance peaks when the total data volume reaches approximately 320GB, which corresponds to 320 tasks running in parallel across 10 nodes. Performance then declines with larger data volumes. This behavior is expected given that the AWS Storage Gateway has a 300GB cache size, and we observe good performance due to cache effects when the working set fits within this limit. Since we focus on peak performance characteristics for subsequent roofline analysis, we do not extend our evaluation beyond 320GB total data volume. The best performance reaches 1130 MiB/sec for writes and 673 MiB/sec for reads.

Figure 1d shows the per-task file size impact. This analysis isolates the effect of individual file size on Storage Gateway performance by scaling per-task file sizes from 4KB to 1000MB while maintaining constant parallelism. From this figure, we aim to identify the optimal file size per task, and we observe that except for

the 4KB file size (with 7 MiB/sec write and 29 MiB/sec read), the remaining file sizes achieve relatively similar read performance, while write performance drops at 1000MB. This suggests that starting at 10MB per task with 320 parallel tasks (1.28GB total I/O size), we can achieve near-maximum throughput from the Storage Gateway.

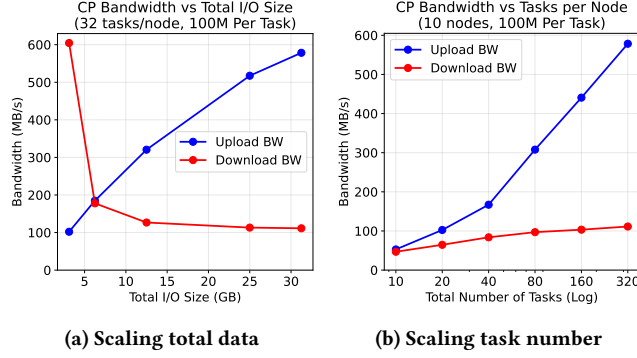


Figure 2: Performance analysis of Data Movement (Method 2) on AWS Gateway

Figure 2a examines how performance scales with increasing total data volume by varying the number of nodes from 1 to 10 while maintaining a constant per-task workload, thus with total I/O size ranging from 3.2GB to 32GB. We observe peak performance when moving data to the AWS Gateway mounted as an NFS path, achieving 580 MiB/sec with a total I/O size of 32GB, while read performance peaks at a 3.2 GB total file size, reaching 604 MiB/sec. This difference in scaling is likely due to the cache on the Storage Gateway. Download performance is high for smaller data volumes that fit within the local cache, but it drops as the cache is bypassed for larger transfers. Upload performance, however, benefits from increased parallelism as the total I/O size grows.

Figure 2b shows how increasing task-level parallelism from 1 to 32 tasks per node affects data movement performance a fixed 10-node configuration with 100MB file I/O per task. We observe that upload bandwidth is similar to that achieved with the 32GB I/O size, while download performance peaks at 104 MiB/sec.

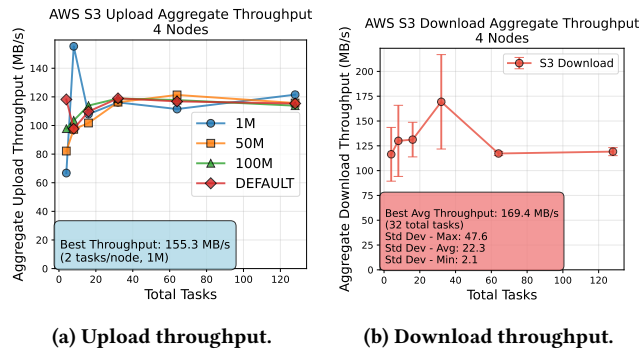


Figure 3: Performance analysis of AWS S3 API (Method 3)

Figures 3a and 3b examine how *Method 3* performance scales with different transfer sizes (1M, 50M, 100M, and DEFAULT) while

varying the total number of parallel tasks across a 4-node configuration, which demonstrates the best overall performance. The analysis focuses on aggregate throughput to understand system-level scalability characteristics rather than per-rank performance metrics. For AWS S3 upload operations, we observe that the 1M transfer size achieves the best performance since it uses AWS S3's more efficient single-file upload strategy rather than multipart uploads (which have a 5MB threshold). Performance also varies significantly with parallelism: for this 4-node configuration, 1 task per node (totaling 4 tasks) performs worst, while 2 tasks per node (totaling 8 tasks) achieves optimal throughput, indicating that moderate parallelism balances resource utilization and coordination overhead. AWS S3 download operations do not vary with transfer size, but we observe large standard deviation in download speed, particularly when tasks are below 60. Download achieves the best average performance with 32 tasks at 169 MB/s. Due to the lack of cache benefits like the storage gateway method, we observe lower performance and higher performance variability.

4.2 Roofline Analysis

In order to analyze the performance of different access methods, we apply the I/O roofline model. However, shown in Section 4.1, neither per-task transfer size nor the total I/O volume play a decisive role in the previous evaluation, and the bandwidth is also already saturated at 10 nodes. Based on these observations, we use only a subset of the previously shown results for the roofline analysis, i.e., results up to 10 nodes and 32 tasks per node (TPN), i.e., 320 parallel tasks in total with a file size of 100 MB and transfer sizes of 1 MB and 100 MB for the three different access methods.

The following diagrams illustrate the read and write performance of three different access methods: NFS-mounted Storage Gateway (POSIX), direct API-based transfers (native-s3), and data migration through Storage Gateway (Copy). Since the transfer to and from S3 occur exclusively over a 10 Gbits Ethernet, in both cases the maximum achievable performance of a 10G Ethernet connection is shown as a roofline.

To determine the maximum achievable I/O operations per second for Ethernet, we derived this metric based on the transfer size/payload size. Since for this work, we look at two different transfer sizes, i.e., 1MB and 100MB, we decided to add three different limits, i.e., small, medium, and large transfer size. For the smallest payload size of 1500 bytes, the theoretical peak IOPS is 833,333 (based on 1250 MB/s / 1500 bytes). For a 1 MB payload, the peak IOPS is 1,250, and for 100 MB, it drops to 12.5. Another assumption is that, for the copy access method, the transfer size equals the file size, resulting a single transfer size of 100MB. The AWS S3 API uses single uploads for files smaller than 5MB and multipart uploads for larger files, where operation count represents the number of upload parts.

The data points show the measured values for specific numbers of nodes and block sizes. Only the best performance is shown, regardless of the number of parallel tasks. For example, for native-s3 write, the best bandwidth for 1MB block size is achieved with 32 tasks per node on 1 node, while for 100MB block size it is achieved with 8 tasks per node on 1 node.

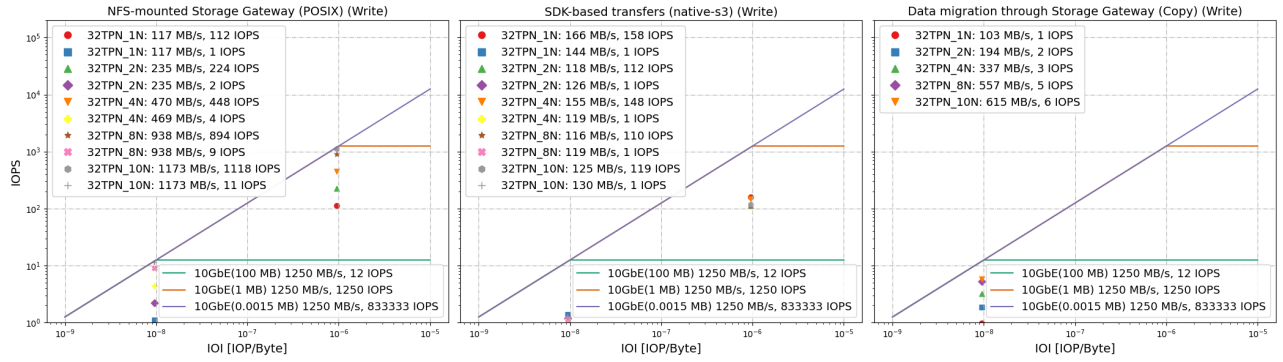


Figure 4: Roofline analysis for write performance up to 10 nodes. The figure depicts how IOI and scale affect IOPS for the three different AWS access methods. The top-left legend indicates the maximum bandwidth and IOPS achieved for different tasks per node and node number configurations. The bottom-right legend shows the theoretical maximum IOPS. Purple lines indicate the maximum theoretical IOPS of the network. Horizontal green and orange lines show the theoretical maximums of different I/O transfer sizes.

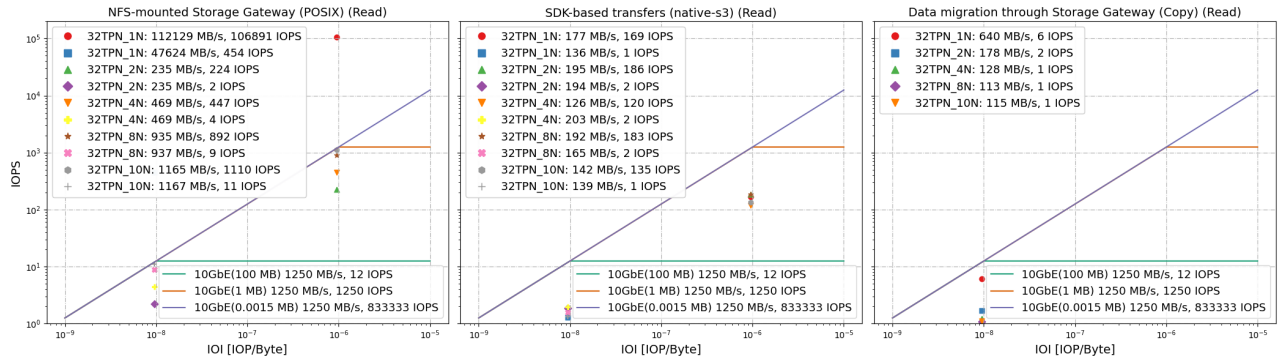


Figure 5: Roofline analysis for read performance up to 10 nodes. Figure layout and legend interpretation are identical to the write analysis.

4.2.1 Write Performance. As shown in Figure 4, in general, performance improves with increasing parallelism in POSIX. It is particularly noticeable that measurements with nearly identical throughput can be achieved using different block sizes. The maximum write bandwidth with POSIX is reached using 320 parallel tasks and can be around 1173 MB/s depending on the transfer size. This is limited by the Ethernet bandwidth and is therefore found just below the corresponding rooflines, indicating an effective utilization of the available bandwidth. The native-s3 write performance is significantly more limited. The highest achieved values are 166 MB/s and 158 IOPS, using 1 node and 32 tasks per node. Compared to POSIX, scaling with higher parallelism plays a less significant role here. The points are clearly below the theoretically possible performance, which indicates overhead or limitations in the protocol or its implementation. For the Copy (from host to server) method, it can also be seen that performance improves with increasing parallelism. The highest performance is reached at 6 IOPS and 615 MB/s using 10 node with 32 tasks per node. Since only the results for 100 MB transfer size were used in the analysis for Copy, it is also evident that the measurement points lie directly below the roofline of the 10G Ethernet line for 100 MB block size, which indicates the same I/O operational intensity.

4.2.2 Read Performance. From Figure 5 we can observe, two outliers stand out in particular for the the performance for POSIX read. While one outlier reaches over 112 GB/s and 106,891 IOPS, the other outlier with a 100 MB transfer size reaches around 47 GB/s and 454 IOPS. Since both outliers are far beyond the realistic limits of a 10G Ethernet connection, this likely indicates a local cache effect. The other setups show similar good performance as for write, i.e., up to nearly 1167 MB/s with 320 parallel tasks, therefore scale with increasing parallelism. Similar to native-S3's write performance, the read performance is also limited under parallel access. A maximum of 203 MB/s and 2 IOPS was reached for a transfer size of 100 MB using 4 nodes, each running 32 parallel tasks. The measured points consistently fall well below the theoretical limits of the network, indicating that the available parallelism cannot be utilized. With a total of 320 parallel tasks, the Copy (from server to host) performance is reaching a maximum of 640 MB/s at 6 IOPS. What stands out is that the best performance was achieved with a single node, and performance decreases with increasing number of nodes. This behavior is in contrast to that of Copy (from host to server). Nevertheless, the measured performance remains far from the Ethernet limits.

4.2.3 Discussion. Overall, for write access, from the results it is clearly noticeable that POSIX shows solid performance, which is however limited by the 10 Gbit Ethernet connection. In contrast, native-s3 shows better performance with a single node but is mainly limited under high parallelism. Copy performance does scale with increasing parallelism, but not as quickly as with access via the POSIX interface. For read access, POSIX offers stable and high read speed, aside from the suspected cache outlier. Both native-S3 and the Copy do not come close to reaching the peak performance of Ethernet, even with up to 10 nodes and 32 tasks per node. Regarding the poor performance observed with native S3, potential reasons could include the additional steps involved in native S3 operations, such as authentication, checksum calculations, and internal request handling, which can introduce significant delays. Additionally, the implementation might not be optimized for high parallelism or may lack efficient connection reuse, leading to excessive setup and teardown overhead per request. Lastly, the underlying S3 backend may throttle or limit performance on a per-client or per-connection basis. Furthermore, since data written via the copy and POSIX access methods is not immediately available on S3 after the function returns, we additionally measured the latency for both methods. For IOR POSIX write on gateway NFS, the average latency is 421.17 seconds, and for data copy on gateway NFS, it is 4,833.5 s. As a result, the performance of these two access methods may be further degraded, especially POSIX write, due to its high latency.

5 Conclusions

Our roofline analysis of three AWS S3 access methodologies reveals distinct performance characteristics. Through extending I/O roofline modeling to cloud storage systems, we demonstrate that NFS-mounted Storage Gateway achieves the highest throughput with high parallelism but incurs significant latency penalties, while direct S3 API transfers show limited scalability due to protocol overhead but provide immediate data availability. Cache effects substantially improve read performance. Our systematic comparison reveals performance differences up to 6.4x between approaches, providing quantitative guidance for practitioners: Storage Gateway excels for high-throughput bulk operations where latency is acceptable, while direct API access suits applications requiring immediate data availability. These findings demonstrate the value of applying I/O roofline modeling to cloud storage and provide actionable insights for optimizing data-intensive scientific workflows in hybrid cloud-HPC deployments.

Acknowledgments

This research is supported by the U.S. Department of Energy (DOE) through the Office of Advanced Scientific Computing Research's "Orchestration for Distributed & Data-Intensive Scientific Exploration" and the "Decentralized data mesh for autonomous materials synthesis" AT SCALE LDRD at Pacific Northwest National Laboratory. PNNL is operated by Battelle for the DOE under Contract DE-AC05-76RL01830. This research is also supported in part by the National Science Foundation under Grants no. CSSI-2104013 and OAC-2313154. This work used TAMU ACES at Texas A&M

University through allocation CHE240191 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

- [1] Daniel Araújo De Medeiros, Stefano Markidis, and Ivy Bo Peng. 2023. LibCOS: Enabling Converged HPC and Cloud Data Stores with MPI. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region* (Singapore, Singapore) (HPCAsia '23). Association for Computing Machinery, New York, NY, USA, 106–116. doi:10.1145/3578178.3578236
- [2] AWS. 2025. SDK General Information. <https://docs.aws.amazon.com/AmazonS3/latest/API/sdk-general-information-section.html>. Accessed: 2025-09-22.
- [3] AWS. 2025. What is Amazon S3 File Gateway? <https://docs.aws.amazon.com/filegateway/latest/files3/what-is-file-s3.html>. Accessed: 2025-09-22.
- [4] David Cardwell and Fengguang Song. 2019. An extended roofline model with communication-awareness for distributed-memory hpc systems. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*. 26–35.
- [5] Fabio Checconi, Jesmin Jahan Tithi, and Fabrizio Petrini. 2022. Ridgeline: A 2D Roofline Model for Distributed Systems. *arXiv preprint arXiv:2209.01368* (2022).
- [6] Jee Whan Choi, Daniel Bedard, Robert Fowler, and Richard Vuduc. 2013. A roofline model of energy. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. IEEE, 661–672.
- [7] Nan Ding, Brian Austin, Yang Liu, Neil Mehta, Steven Farrell, Johannes P Blaschke, Leonid Oliker, Hai Ah Nam, Nicholas J Wright, and Samuel Williams. 2024. A Workflow Roofline Model for End-to-End Workflow Performance Analysis. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [8] Nicolas Dube, Duncan Roweth, Paolo Faraboschi, and Dejan Milojicic. 2021. Future of HPC: The Internet of workflows. *IEEE Internet Computing* 25, 5 (2021), 26–34.
- [9] Frank Gadban and Julian Kunkel. 2021. Analyzing the Performance of the S3 Object Storage API for HPC Workloads. *Applied Sciences* 11, 18 (2021). doi:10.3390/app11188540
- [10] Frank Gadban and Julian Kunkel. 2021. Analyzing the performance of the S3 object storage API for HPC workloads. *Applied Sciences* 11, 18 (2021), 8540.
- [11] Shaoming Guo, Wang Yang, and Guojun Wang. 2013. NFS protocol performance analysis and improvement for mobile transparent computing. In *2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing*. IEEE, 1878–1883.
- [12] Guohua Li, Joon Woo, and Sang Boem Lim. 2021. HPC Cloud Architecture to Reduce HPC Workflow Complexity in Containerized Environments. *Applied Sciences* 11, 3 (2021). doi:10.3390/app11030923
- [13] Richard P Martin and David E Culler. 1999. NFS sensitivity to high performance networks. In *Proceedings of the 1999 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*. 71–82.
- [14] Davit Petrosyan and Hrachya Astsatryan. 2022. Serverless high-performance computing over cloud. *Cybernetics and Information Technologies* 22, 3 (2022), 82–92.
- [15] Rohan Basu Roy, Tirthak Patel, Vijay Gadepally, and Devesh Tiwari. 2022. Mashup: making serverless computing useful for HPC workflows via hybrid execution. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) (PPoPP '22). Association for Computing Machinery, New York, NY, USA, 46–60. doi:10.1145/3503221.3508407
- [16] Lea Shalev, Hani Ayoub, Nafea Bshara, and Erez Sabbag. 2020. Supercomputing on nitro in aws cloud. *IEEE Micro*, vol. PP (2020), 1–1.
- [17] Olivier Terzo and Jan Martinović. 2022. *HPC, Big Data, and AI Convergence Towards Exascale: Challenge and Vision*. CRC Press.
- [18] Simon Triendl and Sashko Ristov. 2024. Peeking Behind the Serverless Implementations and Deployments of the Montage Workflow. In *Companion of the 15th ACM/SPEC International Conference on Performance Engineering* (London, United Kingdom) (ICPE '24 Companion). Association for Computing Machinery, New York, NY, USA, 196–203. doi:10.1145/3629527.3651420
- [19] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [20] Zhaobin Zhu, Niklas Bartelheimer, and Sarah Neuwirth. 2023. An empirical roofline model for extreme-scale i/o workload analysis. In *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 622–627.
- [21] Zhaobin Zhu and Sarah Neuwirth. 2023. Characterization of Large-scale HPC Workloads with non-naïve I/O Roofline Modeling and Scoring. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 737–744.

Appendix: Artifact Description/Artifact Evaluation

A Overview of Contributions and Artifacts

A.1 Paper's Main Contributions

This paper makes three main contributions by extending the I/O roofline model to analyze the performance of AWS S3 storage from High-Performance Computing (HPC) systems. The study compares different access methods, identifies performance bottlenecks, and provides actionable insights for integrating cloud storage into data-intensive scientific computing environments.

- C_1 This work extends the I/O roofline model to characterize AWS S3 performance from HPC systems across three distinct access methods: NFS-mounted Storage Gateway, data migration through Storage Gateway, and direct S3 API transfers.
- C_2 The paper presents a comprehensive comparison of S3 access strategies, analyzing performance for parallel POSIX I/O, data movement efficiency, and direct API transfers under varying data sizes and parallelism levels.
- C_3 The study identifies performance bottlenecks and optimization opportunities for each access method, offering quantitative guidance for S3 integration with observed performance differences of up to 6.4x between approaches.

A.2 Computational Artifacts

This work produces a set of computational artifacts that enable reproducibility of our AWS S3 performance evaluation from HPC systems. These artifacts include benchmark scripts for AWS Storage Gateway (NFS mount and data migration), a custom MPI-parallel C++ benchmark for direct S3 API usage, and the I/O roofline analysis toolkit developed for performance modeling. All artifacts are available in an anonymized GitHub repository.

- A_1 https://anonymous.4open.science/r/ERT4IO-8F41/aws_fuse_bench – Our AWS S3 benchmark scripts (incl. input & outputs) including: benchmark scripts and configurations for AWS Storage Gateway NFS mount evaluation (Method 1), BeeGFS–S3 data migration via Storage Gateway (Method 2), latency analysis scripts measuring the time difference between file creation (via Storage Gateway I/O) and the file's appearance in S3, and custom MPI-parallel C++ benchmark implementation using the AWS SDK for direct S3 API evaluation (Method 3).
- A_2 <https://anonymous.4open.science/r/NeuwirthLab-ERT4IO-PDSW-AB2B/> – I/O roofline analysis scripts and plotting tools (incl. outputs) for performance visualization and bottleneck identification.

The following table summarizes the relation between each artifact and the paper's main contributions. It also indicates which experimental results (figures and tables) can be reproduced using each artifact, covering storage gateway evaluation, direct API benchmarking, and roofline performance modeling.

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3	Tables 1, 2; Figure 1–3
A_2	C_1, C_3	Figure 4; Equations (3)–(6)

B Artifact Identification

B.1 Computational Artifact A_1

B.2 Relation To Contributions

Comprehensive benchmark suite supporting all three main contributions (C_1 – C_3) through experimental infrastructure for I/O roofline modeling, systematic S3 access method comparison, and performance bottleneck identification.

B.3 Expected Results

Demonstrates 6.4× performance differences between S3 access methods. NFS Storage Gateway achieves 1173 MB/s (network-saturated) vs. direct S3 API limited to 166 MB/s. Results quantify trade-offs and provide actionable S3 integration guidance.

B.4 Expected Reproduction Time

Setup: 2–3 hours (AWS Gateway deployment, cluster configuration). Execution: 400+ hours (complete benchmark suite). Analysis: <10 minutes (data processing). Total: 403 hours.

B.5 Artifact Setup (incl. Inputs)

Hardware HPC cluster: 2× AMD EPYC 7502 (64 cores), 264 GB DDR4, 10 GbE network, BeeGFS storage. AWS Storage Gateway with 300 GB cache. AWS connectivity required.

Software IOR benchmark (v4.0.0), Darshan I/O tracing (v3.4.5), AWS SDK for C++ (v1.11.478), MPI implementation (OpenMPI with GCC v11.2.0), NFSv3 client (RHEL 7.8, kernel 3.10.0-1127.18.2), AWS CLI tools (v2.22.28). Custom C++ benchmark included.

To run analysis and recreate plots in the paper, set up the environment using Python 3.10 or higher and run the following commands:

```
# Create a Python 3.10 virtual environment named ert4io_env
python3.10 -m venv ert4io_env
```

```
# Activate the environment
source ert4io_env/bin/activate
```

```
# Upgrade pip inside the environment
pip install --upgrade pip
```

```
# Install dependencies from requirement.txt
pip install -r requirement.txt
```

Datasets / Inputs Dynamically generated test files (4KB–1000MB per task). Transfer sizes: 4KB–100MB. Block sizes: 100MB–1000MB. Supports 1–16 nodes, 1–32 tasks/node configurations.

Installation and Deployment C++ compiler (C++11), MPI libraries, AWS SDK with CMake. AWS Storage Gateway in File Gateway mode with NFS mounting via SSH scripts. AWS credentials required.

B.6 Artifact Execution

Workflow: T_1 (parameter configuration) $\rightarrow \{T_2$ (POSIX I/O), T_3 (data migration), T_4 (direct S3 API) $\rightarrow T_5$ (analysis). Parameters: 320 max tasks, 1MB/100MB transfers, 3 trials, integrity verification. Accounts for asynchronous S3 synchronization latency.

Run the following command to plot Figures 1 – 3:

```
cd aws_fuse_bench/
python results_plot.py
```

B.7 Computational Artifact A_2

B.8 Relation To Contributions

I/O roofline analysis toolkit supporting contributions C_1 and C_3 through mathematical framework implementation for operational intensity calculation and performance boundary visualization.

B.9 Expected Results

Reveals distinct performance characteristics per S3 method. Visualizes POSIX achieving near-theoretical limits (1250 MB/s) vs. direct API remaining below due to protocol overhead. Identifies optimal operational intensity regimes.

B.10 Expected Reproduction Time

Setup: 1 hour (Python environment). Execution: 2-3 hours (processing, plotting). Analysis: 1-2 hours (interpretation). Total: 4-6 hours.

B.11 Artifact Setup (incl. Inputs)

Hardware Standard workstation with 8+ GB RAM for data processing and visualization generation.

Software Python 3.7+ with matplotlib, numpy, scipy, pandas, seaborn. Compatible with standard scientific computing environments.

Datasets / Inputs Benchmark output from A_1 : IOPS measurements, bandwidth calculations, operational intensity values, latency data across all S3 methods and parallelism levels.

Installation and Deployment Standard pip/conda installation. No compilation required. Configurable roofline parameters (1250 MB/s network limits, transfer size categories).

B.12 Artifact Execution

Workflow: T_1 (calculate operational intensity: $\text{IOPS} = \text{Bandwidth} / \text{Transfer Size}$) $\rightarrow T_2$ (generate roofline plots) $\rightarrow T_3$ (comparative analysis). Parameters: theoretical IOPS (833K/1500B, 1.25K/1MB, 12.5/100MB), ridge point identification, 3-trial aggregation.

To execute the artifact, run the following command:

```
python roofline.py
```